

Take-Home Assignment

Baseball Trading · Engineering & Modeling

This assignment has two independent exercises. Complete both. There is no single correct answer to either — we are interested in your reasoning, your handling of edge cases, and the quality of what you build. Document your assumptions wherever the spec is open-ended. Please include a short README describing how to set up and run your code.

EXERCISE 1 · ENGINEERING

Live Fixture Detection & Fair-Value Service

We make markets on Kalshi for MLB game outcomes. On Kalshi, pregame queue priority matters: being first to post an accurate quote when a market opens is a meaningful edge. To quote, we need a continuously updated **sportsbook fair** — the no-vig fair win probability implied by sharp sportsbook odds — for every game as soon as its market exists.

Build the service that powers this. It detects new fixtures the moment they are listed on Kalshi and emits a fair value for each, once per minute, until game time.

Requirements

- Fixture detection.** Detect when a new MLB fixture is listed on Kalshi. Markets list no more than roughly five days in advance. Your service should pick up a new fixture promptly once it appears — define and justify your polling cadence and detection latency. Assume you are competing for queue position, so detection lag is a real cost.
- Fair-value output.** For each detected fixture, every minute until first pitch, emit either:
 - the current **sportsbook fair** (de-vigged moneyline converted to a fair win probability for each side), or
 - the string `"no sportsbook fair"` if no usable book is currently available.
- Book selection and failover.** You may use any sportsbook as your source. Prioritize **Pinnacle** when it is available. However:
 - If Pinnacle is not up yet but another book is, use the best available book until Pinnacle appears, then switch to Pinnacle.
 - Handle individual books going down mid-session gracefully. A book dropping out should never crash the service or cause it to serve a stale fair silently. Define what you consider “stale” and how old a quote you are willing to trust.
 - Make your fallback ordering explicit and justify it.
- Doubleheader handling.** A single matchup (same two teams, same calendar day) can consist of two distinct games (Game 1 and Game 2). Your service must keep these as separate fixtures with separate fairs, and must correctly map each Kalshi market to the corresponding sportsbook game using start times and game numbers. This is the most common place naive implementations break. Also consider: suspended-then-resumed games, split-admission versus traditional doubleheaders, and start-time drift.

Deliverables

- Runnable code, with a README covering setup and how to run it.
- A short design note (one page or less) covering: your detection cadence and why; your failover ordering and why; your definition of stale; and how you map Kalshi fixtures to sportsbook games, especially for doubleheaders.
- Either a live API integration or a mock / recorded feed — your choice. If you mock the feeds, keep the interface realistic so that swapping in a live source is trivial, and document the assumed schema.

You may assume access to a sportsbook odds source (for example, an aggregator that exposes multiple books) and to Kalshi’s public market listing. If you mock these, document the schema you assume.

Live MLB Win-Probability Model

We need a live win-probability estimate for MLB games, updated at each half-inning break. These breaks are the only moments we trade: game state is stable between innings, so you never need to model anything mid-inning. The model feeds our pregame and in-game pricing.

Build a model that, at any half-inning break, outputs the probability that the home team wins, given at minimum the following inputs:

- pregame win probability (home)
- home team and away team
- home team record and away team record
- half-inning (for example, “top 6” or “bottom 6”)
- home score and away score

You may add any additional features that are **easily accessible and effectively zero-latency** — that is, things knowable at the inning break without scraping mid-play data or waiting on a slow source. Reasonable examples include bullpen ERA or reliever quality, whether the starter has been pulled, run differential or Pythagorean expectation, park factors, and rest days. If you add a feature, justify that it is genuinely available with no meaningful latency at the moment you would trade.

Deliverables

- Model code, plus a clear explanation of the functional form you chose and **why** — including how you handle the bounded output and the way the value of a run changes across innings and lead sizes.
- Calibration evidence on a holdout set: a Brier score and a reliability curve (predicted versus realized win rate, bucketed).
- A short note on how the model behaves in edge cases: extra innings, walk-off situations (the home team batting last), and large leads late in the game.
- A discussion of the model’s known blind spots given the inputs available to it.

Please document any assumptions you make. Where the spec leaves a choice open, we want to see the choice you made and the reasoning behind it.